Day 86 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 13, Day 3 reference

Watch: Stanford CS224N, Lecture 1: Intro & Word Vectors (Stanford Online) | Vectoring Words (Word Embeddings) (Computerphile)

`day86_word2vec_skipgram.py`, segment by segment, in the order `main()` uses things. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: c = centre word, o = context word, n = a noise word · v_c = row c of W_in · u_o = row o of W_out · s(z) = $1 / (1 + e^{-z})$ · k = 5 noise words per pair

Segment 0 — Setup

Settings

CODE

```
MIN_COUNT = 5
D = 50
WINDOW = 5
NEG = 5
SUBSAMPLE = 1e-4
EPOCHS = 5
LR0 = 0.025
BATCH = 1024
SEEDS = [0, 1, 2]
```

- `SUBSAMPLE = 1e-4` — the threshold t in the keep probability below; smaller t drops frequent words more.
- `LR0 = 0.025` — word2vec's own starting rate; it falls linearly to almost 0 by the end.

Segment 1 — The corpus and the pairs

load_text8 and build_vocab

CODE

```
def load_text8(path=CORPUS):
    with gzip.open(path, "rt", encoding="ascii") as f:
        return f.read().split()
def build_vocab(words, min_count=MIN_COUNT):
    counts = Counter(words)
    itos = [w for w, c in sorted(counts.items(), key=lambda x: (-x[1], x[0])) if c >= min_count]
    stoi = {w: i for i, w in enumerate(itos)}
    freq = np.array([counts[w] for w in itos], dtype=float)
    ids = np.array([stoi[w] for w in words if w in stoi])
    return itos, stoi, freq, ids
```

- `gzip.open(path, "rt", encoding="ascii")` — opens the compressed file as text: decompression happens while reading.
- `sorted(counts.items(), key=lambda x: (-x[1], x[0]))` — most frequent first; equal counts in alphabetical order, so the ids never change between runs.
- `np.array([stoi[w] for w in words if w in stoi])` — the corpus as ids, rare words simply left out.

subsample: dropping frequent words at random

MATH

$$keep(w) = min(1, \sqrt{(t / f(w))} + t / f(w)), \quad f(w) = count(w) / tokens$$

CODE

```
def subsample(ids, freq, t, rng):
    f = freq[ids] / len(ids)
    keep = np.minimum(1.0, np.sqrt(t / f) + t / f)
    return ids[rng.rand(len(ids)) < keep]
```

- `freq[ids] / len(ids)` — fancy indexing: each token's word frequency, one number per token.
- `ids[rng.rand(len(ids)) < keep]` — a boolean mask: each token survives with its own probability.

skipgram_pairs: every (centre, context) pair at once

CODE

```
def skipgram_pairs(ids, window, rng):
    b = rng.randint(1, window + 1, size=len(ids))
    C, O = [], []
    for off in range(1, window + 1):
        right = b[:-off] >= off
        C.append(ids[:-off][right]); O.append(ids[off:][right])
        left = b[off:] >= off
        C.append(ids[off:][left]); O.append(ids[:-off][left])
    return np.concatenate(C), np.concatenate(O)
```

- `b = rng.randint(1, window + 1, size=len(ids))` — each centre's own window size, so near words are paired more often than far ones.
- `ids[:-off][right], ids[off:][right]` — slicing by an offset pairs every token with the one off places later, all at once, with no Python loop over tokens.
- `np.concatenate(C)` — joins the per-offset arrays into one long array of centres.

noise_cdf: drawing noise words

MATH

$$P(noise\ word = w) = count(w)^{0.75} / \Sigma\ count^{0.75}$$

CODE

```
def noise_cdf(freq):
    p = freq ** 0.75
    return np.cumsum(p / p.sum())
```

- `np.cumsum(p / p.sum())` — the cumulative distribution; np.searchsorted(cdf, rng.rand(...)) then draws words with these probabilities.

Segment 2 — The two objectives

sgns_loss_grads: the negative-sampling loss

MATH

$$loss = -\log s(u_o \cdot v_c) - \Sigma_n \log s(-u_n \cdot v_c)$$

$$dloss/dz = s(z) - 1 \text{ (real pair)}, s(z) \text{ (noise)}; \quad dv = \Sigma\, dz \cdot u, \quad du = dz \cdot v$$

CODE

```
def sigmoid(z):
    return np.exp(-np.logaddexp(0, -z))
def sgns_loss_grads(Win, Wout, c, tgt):
    v, U = Win[c], Wout[tgt]
    z = np.einsum("nd,nkd->nk", v, U)
    loss = np.logaddexp(0, -z[:, 0]).sum() + np.logaddexp(0, z[:, 1:]).sum()
    gz = sigmoid(z); gz[:, 0] -= 1
    dv = np.einsum("nk,nkd->nd", gz, U)
    dU = gz[:, :, None] * v[:, None, :]
    return loss, dv, dU
```

- `np.exp(-np.logaddexp(0, -z))` — logaddexp(0, a) = log(1 + e^a) without overflow, so this is a safe sigmoid.
- `np.einsum("nd,nkd->nk", v, U)` — a dot product per (pair, candidate): n pairs, k + 1 candidates each.
- `gz[:, 0] -= 1` — s(z) – 1 for the real pair (column 0); s(z) for every noise word.

softmax_loss_grads: the full softmax it replaces

MATH

$$p(o \mid c) = exp(u_o \cdot v_c) / \Sigma_w exp(u_w \cdot v_c) \quad (V \text{ terms in the sum})$$

CODE

```
def softmax_loss_grads(Win, Wout, c, o):
    v = Win[c]
    z = v @ Wout.T
    z = z - z.max(axis=1, keepdims=True)
    logp = z - np.log(np.exp(z).sum(axis=1, keepdims=True))
    loss = -logp[np.arange(len(c)), o].sum()
    gz = np.exp(logp); gz[np.arange(len(c)), o] -= 1
    return loss, gz @ Wout, gz.T @ v
```

- `z = v @ Wout.T` — a score for every vocabulary word: (n, V) numbers per batch, which is the cost.
- `gz.T @ v` — the gradient for the whole output table at once: every word's row moves on every step.

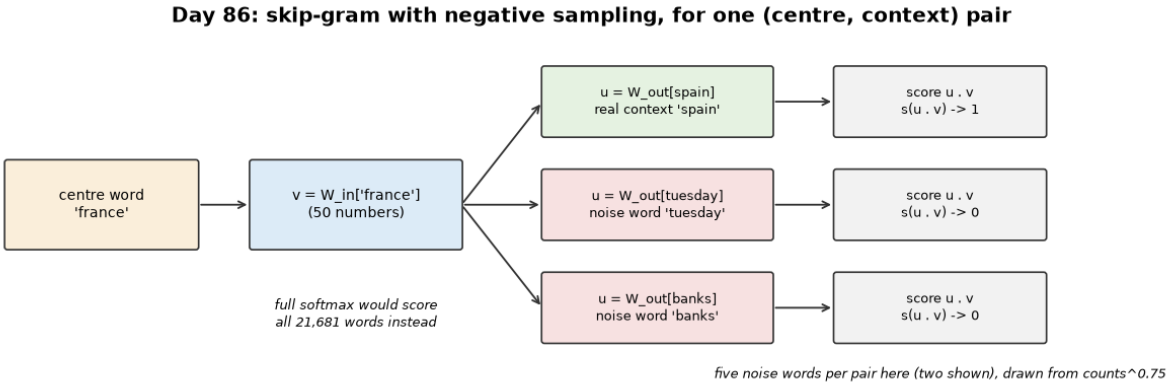
SkipGram and train

CODE

```
class SkipGram:
    def __init__(self, V, d=D, seed=0):
        rng = np.random.RandomState(seed)
        self.Win = (rng.rand(V, d) - 0.5) / d
        self.Wout = np.zeros((V, d))
    def step(self, c, tgt, lr):
        loss, dv, dU = sgns_loss_grads(self.Win, self.Wout, c, tgt)
        np.add.at(self.Win, c, -lr * dv)
        np.add.at(self.Wout, tgt, -lr * dU)
        return loss
def train(words_ids, freq, V, seed, window=WINDOW, neg=NEG, t=SUBSAMPLE, epochs=EPOCHS, itos=None,
    stoi=None):
    rng = np.random.RandomState(seed + 100)
    model = SkipGram(V, seed=seed)
    cdf = noise_cdf(freq)
    total, done = epochs * len(words_ids), 0
    hist = []
    for ep in range(epochs):
        kept = subsample(words_ids, freq, t, rng) if t > 0 else words_ids
        C, O = skipgram_pairs(kept, window, rng)
        perm = rng.permutation(len(C)); C, O = C[perm], O[perm]
        loss = 0.0
        for s in range(0, len(C), BATCH):
            c, o = C[s:s + BATCH], O[s:s + BATCH]
            lr = LR0 * max(1e-4, 1 - (done + s / len(C) * len(words_ids)) / total)
            tgt = np.concatenate([o[:, None], np.searchsorted(cdf, rng.rand(len(c), neg))],
                axis=1)
            loss += model.step(c, tgt, lr)
        done += len(words_ids)
        hist.append((loss / len(C) / (1 + neg),) + evaluate(model.Win, itos, stoi))
    return model, np.array(hist)
```

- `np.add.at(self.Win, c, -lr * dv)` — a word used by several pairs in one batch collects all their updates (Day 85).
- `np.searchsorted(cdf, rng.rand(len(c), neg))` — draws neg noise words for every pair at once.
- `hist.append((...,) + evaluate(...))` — the scores after every epoch, not only at the end.

SYSTEM DIAGRAM



One (centre, context) pair and two of its five noise words.

Segment 3 — Measuring

category_score and analogy_score

MATH

$$answer(a{:}b :: c{:}?) = \operatorname{argmax}_w \cos(w, b - a + c)$$

CODE

```
def category_score(E, stoi):
    words = [(w, c) for c, ws in CATEGORIES.items() for w in ws]
    ii = np.array([stoi[w] for w, _ in words]); cc = np.array([c for _, c in words])
    N = unit(E[ii]); S = N @ N.T; np.fill_diagonal(S, -np.inf)
    return np.mean(cc[S.argmax(axis=1)] == cc)
def analogy_score(E, pairs, itos, stoi):
    N = unit(E); ok = n = 0
    for a, b in pairs:
        for c, d in pairs:
            if a == c: continue
            q = N[stoi[b]] - N[stoi[a]] + N[stoi[c]]
            s = N @ q; s[[stoi[a], stoi[b], stoi[c]]] = -np.inf
            ok += itos[int(s.argmax())] == d; n += 1
    return ok / n
```

- `np.fill_diagonal(S, -np.inf)` — a word is not allowed to be its own nearest neighbour.
- `s[[stoi[a], stoi[b], stoi[c]]] = -np.inf` — the three question words cannot be the answer (the usual rule).
- `itos[int(s.argmax())] == d` — right only if the single nearest word is the expected one.

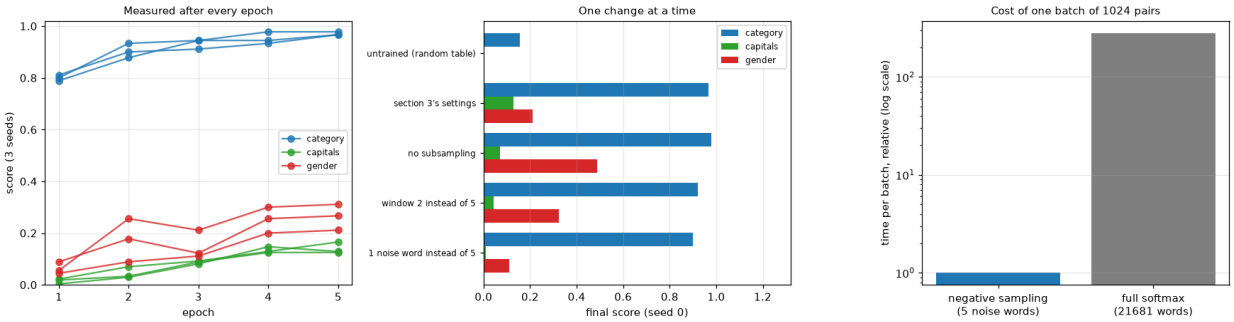
save_vectors: word2vec's text format

CODE

```
def save_vectors(E, itos):
    path = os.path.join(HERE, "day86_vectors.txt.gz")
    with gzip.open(path, "wt", encoding="ascii") as f:
        f.write(f"{len(itos)} {E.shape[1]}\n")
        for i in range(len(itos)):
            f.write(itos[i] + " " + " ".join(f"{x:.4f}" for x in E[i]) + "\n")
```

- `f.write(f"{len(itos)} {E.shape[1]}\n")` — the header: how many words, how many numbers each.
- `" ".join(f"{x:.4f}" for x in E[i])` — four decimals per number keeps the file small.

REAL GRAPH



Real graph from the script: scores by epoch, one change at a time, and the cost of one batch.

Quick Syntax Glossary — new or reinforced constructs

- `gzip.open(path, "rt")` — read a compressed file as text.
- `np.logaddexp(0, z)` — $\log(1 + e^z)$, computed without overflow.
- `np.searchsorted(cdf, u)` — where each u falls in a cumulative distribution: sampling from it.
- `np.einsum("nd,nkd->nk", v, U)` — many dot products in one call.
- `np.add.at(A, idx, X)` — add rows into A, adding up repeated indices.
- `a[:-k], a[k:]` — two shifted views: every element paired with the one k later.

End of Day 86 syntax reference. For the experiments and what they showed, see the Day 86 complete guide.